

RadioCountToLeds仿真实验流程与程序分析

实验整体流程

1. 安装TinyOS开发环境并编译仿真程序：首先需要安装TinyOS 2.x的开发环境以及TOSSIM仿真器支持。安装完成后，在TinyOS提供的示例中找到 RadioCountToLeds 工程，并使用仿真目标进行编译（例如运行 `make micaz sim`）。这将生成仿真所需的库和文件，包括TOSSIM Python模块（如 `_TOSSIMmodule.so`）和消息类型的Python/Java类（例如 `RadioCountMsg.py` 和 `RadioCountMsg.java`），供后续仿真脚本使用。编译成功后，得到可用于仿真的应用程序镜像。

2. 编写并配置仿真脚本：在实验中，需要编写一个Python仿真脚本（如用户提供的 `test.py`），利用TOSSIM接口模拟至少三个节点的无线传感器网络通信^①。脚本中首先通过TOSSIM API载入仿真环境，读取网络拓扑文件（例如 `topo.txt` 定义节点间的无线链路及信号衰减）和噪声文件（如 `meyer-heavy.txt` 用于模拟环境噪声），然后添加无线信道模型和噪声模型到各个节点。接下来，在脚本中创建多个节点实例（节点ID例如1, 2, 3），并设置它们的启动时间（通过 `bootAtTime()` 错开引导时间，以模拟不同节点的不同时刻启动）。最后，将需要监控的调试信道（如应用程序的 `RadioCountToLedsC` 调试信息和系统 `Boot` 信息）重定向输出到日志文件（如 `sim.log`），然后运行仿真一定的事件步数。

3. 运行仿真并记录日志：执行仿真脚本，TOSSIM将按照设定的拓扑和噪声环境模拟节点的启动、定时器中断、无线收发等事件。在仿真过程中，每当节点发送或接收到数据包，应用程序会通过调试接口打印日志信息，TOSSIM捕获这些信息并写入预先指定的日志文件。日志详细记录了每个节点发生的重要事件的节点ID、事件内容、数据包计数以及发生的仿真时间戳。仿真运行预定的事件数量（例如 `runNextEvent()` 循环若干次）后结束，并关闭日志文件以保存结果。

4. 分析日志计算指标：仿真结束后，需要根据日志数据计算网络性能指标，包括数据包的收包率和平均时延^②。具体做法是解析日志文件中记录的每次“请求”发送和“回应”接收事件，匹配对应的数据包，然后统计成功收发的比例和发送到接收之间的时间差。收包率按照实验要求定义为“收到回应的数据包个数 / 发出请求的数据包个数”^③；平均时延定义为“从发出请求数据包到收到回应数据包的时间差”的平均值^③。这些指标可以通过编写脚本（如 `test.py` 内置的日志分析函数）自动计算，并将结果输出呈现。

主要程序逻辑解析

RadioCountToLeds NesC程序逻辑

RadioCountToLeds 应用由两个NesC文件组成：**配置组件**（`RadioCountToLedsAppC.nc`）和**模块组件**（`RadioCountToLedsC.nc`）。配置组件 `RadioCountToLedsAppC` 负责将模块所用接口连接到TinyOS平台提供的具体实现。例如，它将 `RadioCountToLedsC` 模块的**计时器接口**连接到 `TimerMilliC`（毫秒定时器），**无线发送接口**连接到 `AMSenderC(AM_RADIO_COUNT_MSG)`，**无线接收接口**连接到 `AMReceiverC(AM_RADIO_COUNT_MSG)`，**收发控制接口**连接到 `ActiveMessageC`，**LED接口**连接到 `LedsC` 等。这里 `AM_RADIO_COUNT_MSG` 是定义在 `RadioCountToLeds.h` 中的消息类型ID，用于标识该应用发送的消息类型（值为6）。通过这些接口连接，应用程序可以使用高层接口（如 `AMSend.send` 发送消息，`Receive.receive` 接收消息，`Timer<fired>` 定时触发等），而具体的底层实现由TinyOS框架提供。

模块组件 `RadioCountToLedsC.nc` 包含该应用的具体逻辑实现。它定义了一个消息缓冲区（`message_t packet`）、一个16位计数器变量（`uint16_t counter`）和一个锁标志（`bool locked`）用于控制发送节奏。其工作流程如下：

- **系统启动 (Boot):** 每个节点上电后，首先触发 `Boot.booted` 事件。程序在这个事件中启动主动消息层（无线收发组件），调用 `AMControl.start()` 打开无线射频模块。
- **无线模块启动完成:** 当无线收发组件初始化完成后，会回调 `AMControl.startDone` 事件。如果启动成功（`err == SUCCESS`），则开始应用程序主循环的定时器，调用 `MilliTimer.startPeriodic(250)` 设置一个周期为250毫秒的定时器（即4Hz频率）。这个定时器周期性地触发 `MilliTimer.fired` 事件。若启动不成功，则重试调用 `AMControl.start()` 直到成功为止。
- **定时器触发发送 (Timer.fired):** 每当定时器触发时，节点执行 `MilliTimer.fired` 事件处理代码。它首先自增本节点的发送计数器 `counter`，然后检查发送锁 `locked` 是否为FALSE以确保上一次数据包发送已完成。如果未锁定，则准备发送新的数据包：调用 `Packet.getPayload` 获取预分配的 `message_t packet` 中的负载缓冲区指针，填充其中定义的 `radio_count_msg_t` 结构的字段（将 `rcm->counter` 设置为当前计数值）。接下来调用 `AMSend.send(AM_BROADCAST_ADDR, &packet, sizeof(radio_count_msg_t))` 广播发送该数据包至网络中。TinyOS采用非阻塞发送，`send` 调用会立即返回发送请求是否成功入队。若返回 `SUCCESS`，则代表发送已开始执行，于是程序将 `locked` 标志设为TRUE，表示发送队列正忙，同时通过调试接口打印一行日志，标记“packet sent”以及该包的计数值和当前仿真时间戳。这条日志记录了节点发出请求数据包的时刻和序号。
- **发送完成 (SendDone):** 当底层无线发送完成后（无论成功发送并收到ACK与否），会触发 `AMSend.sendDone` 事件。程序在此事件中检查传回的消息指针是否就是先前发送的 `&packet`，如果是则将 `locked` 标志重置为FALSE，表示可以发送下一包数据了。这样在下一个定时周期到来时，新的数据包才会发送，从而保证每次只有一个未完成发送在进行，防止发送队列堆积。
- **接收数据 (Receive.receive):** 当节点收到一个无线数据包时，触发 `Receive.receive` 事件处理函数。函数首先检查数据长度是否符合 `radio_count_msg_t` 结构长度，若匹配则将收到的数据负载转换为该结构指针，然后读取其中的计数值 `rcm->counter`。每当成功接收一个数据包，程序通过调试接口打印一行日志，记录“received packet”以及收到的计数值和当前仿真时间戳。这条日志标记了节点收到回应数据包的时刻和序号。此外，应用还根据收到的计数值更新本节点LED灯的状态：将计数值的最低3个比特映射到三颗LED的亮灭状态（例如counter最低位决定LED0亮灭，以此类推）。这一LED显示部分验证了节点确实获取了发送方的数据内容（不同计数值会点亮不同组合的LED）。处理完成后，`Receive.receive` 返回传入的消息缓冲区指针，以供系统重复利用。

综上，NesC程序的核心逻辑是：每个节点周期性广播一个自增计数值消息，且任何节点收到消息后都会在本地记录日志并更新LED显示。这一简单的协议实现了节点之间单跳广播通信，也为后续统计收包率和时延提供了基础数据（通过日志中的时间戳和计数序号来对应发送和接收事件）。

TOSSIM仿真脚本逻辑

Python仿真脚本 `test.py` 用于设置网络仿真环境、执行模拟并解析日志计算结果。其逻辑可分为仿真初始化和日志分析两部分：

- **仿真初始化与运行:** 脚本首先导入TOSSIM库并创建仿真对象，如 `tos = Tossim([])`。然后通过 `tos.radio()` 获取无线电信道模拟器 `radio`，并读取拓扑配置文件 `topo.txt` 逐行添加链路：每一行拓扑文件通常包含三个值“源节点ID 目标节点ID 信号增益(dBm)”，脚本调用 `radio.add(src, dest, gain)` 建立对应的链路模型。接着，脚本打开噪声文件 `meyer-heavy.txt`，将其中的噪声读数加载到每个节点的噪声模型：对于每个将要模拟的节点（ID 1,2,3），调用 `node.addNoiseTraceReading(value)`

读入一系列噪声样本，再调用 `node.createNoiseModel()` 生成噪声环境。这样，仿真时无线信号传播会受到噪声干扰，以更真实地模拟丢包现象。

随后，脚本准备日志记录文件，例如打开 `sim.log` 供写入。通过调用 `tos.addChannel('RadioCountToLedsC', log_f)` 等，将前述NesC程序中使用的调试信道加入监听列表，使对应的 `dbg()` 输出重定向到日志文件中。脚本设置两个调试信道：一个是 `RadioCountToLedsC`（应用程序发送/接收事件的日志），另一个是 `Boot`（节点启动事件日志）。这样，仿真过程中这些重要事件都会被记录。

完成环境配置后，脚本通过 `tos.getNode(n)` 获取节点实例并调用 `bootAtTime(t)` 启动每个节点。例如脚本中将节点1在仿真时间100001触发启动，节点2在800008时启动，节点3在1800009时启动（这些时间值单位为仿真“ticks”，TOSSIM默认每秒4百万tick）。错开引导时间可以避免所有节点完全同步发送，模拟更真实的异步网络。启动节点后，脚本执行 `run_sim(tos, ev=50000)`，循环调用 `tossim.runNextEvent()` 来处理仿真事件队列中的事件，总计执行50000个事件步骤。这个过程中，各节点按照NesC程序逻辑产生发送/接收等事件，调试信息被写入日志文件 `sim.log`。当指定数量的事件处理完毕后，脚本关闭日志文件，结束模拟运行。

- **日志解析与指标计算：** 仿真结束后，脚本定义并调用 `analyse('sim.log')` 函数，对生成的日志文件进行分析处理。分析的思路是遍历日志中的每一行，利用正则表达式提取发送和接收事件的信息：脚本预先定义了两个正则模式 `SEND_RE` 和 `RECV_RE`，分别匹配日志行中的“packet sent”记录和“received packet”记录，从中提取数据包的计数值（`counter`）和时间戳（`time`）。同时还定义了模式 `NODE_RE` 用于捕获日志行前缀的节点ID（如 `DEBUG (1)` 中的 `1`）。在解析过程中，脚本维护计数器和时间的映射关系：对于每一条发送记录，将其计数值和发送时间tick入队列保存；对于每一条接收记录，则根据计数值查找对应的发送时间，计算二者差值得到该包的传输延迟，并将延迟加入列表。具体而言，脚本使用了一个字典 `send_q`，其中每个键是消息的计数值，值是一个队列，用于存放按时间顺序出现的多个发送事件（如果计数值多次循环使用时可能会出现多个发送）。当处理到接收事件时，脚本查看相应 `counter` 的发送队列是否非空，如果有则弹出最早的一条发送时间，与当前接收时间相减得到一个延迟值（单位为仿真tick），然后换算成毫秒加入延迟列表。同时脚本统计发送事件总数 `sends` 和接收事件总数 `recvs`。这种队列匹配方式可确保每次发送至多只匹配到一次接收，用“先到先服务”的策略避免一个发送被多次接收重复配对，或者多个发送与一个接收混淆。

日志遍历完成后，脚本根据统计值计算收包率PDR和时延统计结果。由于本实验采用广播通信，单次发送理论上会被网络中其它 `N-1` 个节点接收到，因此脚本将**期望接收数**定义为 `expected = sends * (n_nodes - 1)`（其中 `n_nodes` 为参与仿真的节点总数）。例如3个节点互相广播的情况下，每发出1个包，理想情况下有另外2个节点能收到，故期望接收数=发送数×2。随后计算PDR为实际接收数 `recvs` 除以期望接收数 `expected`。这样得到的PDR即表示网络整体的平均收包比例（考虑所有可能接收的链路总计）。同时，脚本对收集的所有延迟值求平均值，并找出最小和最大延迟，作为统计的时延指标输出。最后，`test.py` 打印总结，包括发现的节点数、总发送包数、总接收包数、期望接收数以及计算得到的PDR百分比和平均/最小/最大延迟（毫秒）。这些结果直观反映了仿真网络的通信可靠性和时效性。

各任务的详细实现方式

数据发送任务的实现

在本实验中，各节点周期性执行数据发送任务。具体由NesC应用的定时器事件触发：每隔250ms（4Hz），`MilliTimer.fired` 事件被触发，节点就准备并发送一个数据包。发送任务的实现包含以下步骤：

1. **准备数据并检查状态：** 定时器触发时，节点先将本地计数器 `counter` 加1，表示即将发送第`counter`次数据包。随后检查标志 `locked` 是否为TRUE。如果上一轮发送尚未完成（`locked`为TRUE），则此次发送任务直接返回取消，避免并行发送冲突；若 `locked` 为FALSE，则继续进行发送准备。
2. **封装消息：** 节点调用 `Packet.getPayload(&packet, sizeof(radio_count_msg_t))` 获取消息缓冲区中可写入负载的指针，并将计数值填入该负载结构（即 `rcm->counter = counter`）。`radio_count_msg_t` 结构在 `RadioCountToLeds.h` 中定义，仅包含一个16位的计数字段，足以存储发送次数计数。
3. **广播发送：** 调用 `AMSend.send(AM_BROADCAST_ADDR, &packet, sizeof(radio_count_msg_t))` 执行非阻塞发送。

AM_BROADCAST_ADDR 是TinyOS定义的广播地址（通常0xffff），因此消息会广播给所有邻居节点。发送调用立即返回结果，如果返回 SUCCESS 表示发送请求已成功提交至底层（并不保证空中一定送达）。此时程序进入发送后处理：- 将 locked 置为TRUE，防止定时器下一次触发时再次发送（直到当前发送完成解除锁定）。- 记录日志：通过 dbg 调试接口输出“packet sent”消息，内容包括当前计数值和仿真当前时间。仿真时间戳由 sim_time() 获取，表示自仿真开始以来的时间tick计数。日志输出例如：DEBUG (1): packet sent. counter=1 time=2441506351（表示节点1在仿真时刻2441506351发送了计数为1的包）。该日志为后续计算提供了“请求发送”的时间记录。

1. **发送完成解锁**：对于每一次发送，TinyOS底层都会在发送过程结束时产生 sendDone 事件。应用在 AMSend.sendDone 事件中对传入的消息缓冲区指针是否为本模块的 packet，如果是且此前locked为TRUE，则将 locked 复位为FALSE，表示发送过程结束、可以发送下一个数据包。这样实现了一个简单的流控：保证每个周期至多发送一个包且不重叠。

通过上述机制，实现了**定时发送**：各节点以固定周期广播各自的计数值。使用锁机制避免重复发送，并利用调试日志详细记录每次发送行为（包括发生的节点、计数和时间）。

数据接收与节点通信

由于采用广播通信，网络中每个节点既可能是发送者也可能作为接收者。节点通信的实现由TinyOS主动消息组件处理无线收发，应用层通过 Receive.receive 事件获取收到的数据。具体如下：- 当节点A发送出广播包后，所有在通信范围内的节点（例如邻居节点B、C）都会在稍后几毫秒内收到该包。TOSSIM根据设定的拓扑增益和噪声模型来决定每个邻居是否成功接收（如果信号强度不足或被噪声淹没则可能丢包）。每当某节点成功接收到一个数据帧，TinyOS随即在该节点上触发应用的 Receive.receive 事件。- 在 RadioCountToLedsC 的 Receive.receive 实现中，首先通过参数获取收到的数据负载及长度，并判断长度是否等于 radio_count_msg_t 结构长度。如果匹配，则将负载指针转换为 radio_count_msg_t 类型，以读取其中的计数字段 rcm->counter。因为本应用协议简单，所有收到的都是广播计数消息，没有额外的区分。- 应用随后执行**日志记录**和**LED更新**两项操作：

(1) 调用 dbg 接口输出“received packet”日志，记录收到的数据包计数值及当前仿真时间戳。例如：DEBUG (3): received packet. counter=1 time=2480963456 表示节点3在仿真时刻2480963456收到了计数为1的一个数据包（可以推断出这是某节点在稍早前发送的counter=1包）。多个节点若几乎同时收到同一包，则会各自记录各自的接收时间，这些接收时间一般略有差异，视TOSSIM调度顺序可能先后。

(2) 根据计数值更新LED显示：应用按收到的 counter 值的最低三位控制LED0、LED1、LED2的亮灭。这一部分实现了实验要求中“收到数据后节点做出反应”的逻辑，在物理环境中LED状态的改变可直观反映收到的数据值。在仿真中虽然无法“看到”LED，但这一功能证明通信确已发生且数据内容正确接收。

Receive.receive 最后返回消息缓冲区指针 bufPtr，表示应用层处理完毕，该消息可以由系统释放或复用。TinyOS还提供自动ACK等机制（本应用未使用ACK），这里的返回值不影响默认行为。

值得注意的是，由于所有节点都运行相同代码，默认情况下每个节点也会周期性发送广播包。因此在三节点网络中，可能出现“节点1发送，2和3接收；节点2发送，1和3接收；节点3发送，1和2接收”的循环局面。每个发送包都可能被多个邻居收到，形成一对多的通信。在日志中我们可以看到，一次发送对应后续多个节点的接收记录。例如节点1发送 counter=1 后，日志中紧随其后的接收记录里，节点3和节点2都记录了收到该包（计数1）的时间。这种**广播式节点通信**使得数据在网络中扩散开来，可用于模拟诸如广播公告、邻居发现等场景。实验通过统计**发送-接收日志**来评估整体通信有效性，即后面将计算的收包率PDR反映了广播在邻居间成功传递的比例。

日志记录机制

本实验高度依赖日志来记录和分析节点间通信过程，因此日志记录机制的实现值得详述。日志分为两部分：一是在NesC应用中调用 dbg 接口打印事件信息，二是TOSSIM将这些打印重定向保存至文件。

在NesC代码中, 使用了 `dbg("RadioCountToLedsC", " ... ", args...)` 来打印重要事件。其中 "RadioCountToLedsC" 是自定义的调试信道名字, TOSSIM允许按信道筛选输出。代码在以下几个关键点打印日志: - **发送时**: 在成功调用 `AMSend.send` 后, 打印 `"packet sent. counter=%hu time=%" PRIu64 "\n"`, 即当前包序号和时间戳。- **接收时**: 在 `Receive.receive` 中, 打印 `"received packet. counter=%hu time=%" PRIu64 "\n"`, 即收到包的序号和时间戳。- **调试信息**: 代码中还打印了 `"timer fired, counter is %hu. \n"` 等调试信息, 以跟踪每次定时器触发时计数值的变化。这些信息有助于理解程序流程, 但对收包率/时延统计无直接影响, 所以分析时通常忽略此类行。

打印函数内部使用 `sim_time()` 获取当前64位仿真时钟值 (tick), 并配合格式化宏 `PRIu64` 打印出完整的数值。这些tick值可用于精确计算事件间的时间差。由于TOSSIM仿真时钟默认频率4MHz, 每1秒=4,000,000 ticks, 因此两个时间戳之差除以4e6再乘1000即可换算成毫秒级延迟。

TOSSIM仿真脚本通过 `tos.addChannel("ChannelName", file)` 开启指定信道的日志输出。在 `test.py` 中, 开启了 `RadioCountToLedsC` 和 `Boot` 两个信道, 并将输出定向到打开的 `sim.log` 文件。因此, 当仿真运行时, 这两个信道的 `dbg` 输出都会被写入 `sim.log`。需要注意, 如果不addChannel某信道, 则对应的 `dbg` 信息不会出现在输出中。这里选择记录 `Boot` 信道, 可以获取节点启动的相关信息; 记录 `RadioCountToLedsC` 信道, 则捕获我们关心的发送/接收事件日志。实验要求附加说明中提到“仿真过程记录到log日志中”⁴ 正是通过这种机制实现的。

打开日志文件后, 仿真脚本按顺序执行事件, 日志文件中逐渐积累记录。日志每行通常格式如: `DEBUG (nodeID): ...`, 其中 `nodeID` 是打印该行日志的节点编号, 后面的内容就是 `dbg` 调用提供的信息文本。例如一条实际日志行:

```
DEBUG (2): received packet. counter=5 time=26608474311
```

含义是“节点2: 收到数据包, 计数=5, 时间=26608474311”。通过日志, 我们可以直观地跟踪网络中的消息流动: 哪个节点何时发出了多少次消息, 以及哪些节点收到了这些消息。这些数据为后续计算**收包率**和**时延**提供了基础。

收包率与时延的计算方法

根据记录在日志中的发送和接收事件时间, 我们可以计算网络通信的收包率和时延等性能指标。

- **收包率 (PDR)**: 定义为收到的回应数据包数量与发出的请求数据包数量的比值³。在本实验中, 由于每个发送的数据包并非面向特定单个接收者, 而是广播给周围所有节点, 因此我们在统计PDR时考虑总体的传输成功率。仿真脚本的计算方式是: 首先统计总发送包数 `sends` 和总接收包数 `recvs` (所有节点合计)。假设网络中有N个节点, 那么每次发送理论上会有N-1个可能接收方, 因此**期望接收总数** `expected = sends * (N-1)`。实际PDR则计算为 `recvs / expected`。举例来说, 3节点网络下如果总共发送了100个包, 且每包有2个潜在接收者, 则期望接收数为200; 若实际接收到180个包, 则 $PDR = 180/200 = 90\%$ 。PDR结果通常以百分比表示, 用于衡量网络整体的可靠传输比例。
- **时延 (Latency)**: 这里指单跳传输时延, 定义为数据包从发送时刻到被接收方收到的时间差。³ 由于每条广播消息可能被多个节点收到, 我们通常关注每个**发送-接收**对的时延。计算方法是在日志中找到某一发送事件及其对应的接收事件, 计算接收时间减去发送时间得到该链路上传输的延迟。如上文所述, `test.py` 通过匹配日志里的 `counter` 序号来关联发送和接收。当一个发送序号X的消息被k个节点收到时, 将产生k条接收记录。为了避免歧义, 脚本选择将**每次发送只匹配一个接收**来计算时延 (实现中是为每个计数值维护一个FIFO队列, 遇到接收记录就从队列中弹出一个发送时间进行配对)。这样得到的延迟近似代表每次消息传播到首个收到节点的时间。尽管这种简化可能忽略部分接收 (如果一个发送被多个节点收到, 只有第一条配对的延迟被记录), 但由于单跳广播延迟在理想情况下对所有同时收到

的节点应当接近，相差不大，因此取首个也具有代表性。脚本将所有成功配对的发送-接收延迟累计进列表，然后计算出**平均时延**、**最小时延**和**最大时延**。结果通常以毫秒为单位表示。例如输出“Average delay = 4.532 ms”，表示网络中消息单跳传输平均耗时约4.532毫秒。

值得注意的是，该实验中的时延基本反映MAC层在无严重拥塞情况下的广播延迟（包含处理和传播延迟）。由于节点非常接近（仿真拓扑中信号强度配置较高），没有多跳路由和排队拥塞，因此时延值应当很小且稳定在几个毫秒以内。若出现某些延迟明显偏大，可能是因为脚本简单配对机制导致了跨序号错误匹配。在提供的 `test.py` 中，作者也通过调整算法（使用每个counter值的最近发送匹配接收）来尽量避免计算异常。

最后，`test.py` 会将**收包率**和**时延统计**打印出来，作为实验结果的一部分。例如，日志分析的结果可能显示节点数、发送/接收包总数，以及 PDR: 95.00%、平均时延 4.5 ms 等信息。这些数据可以帮助我们评估网络性能：收包率接近100%说明绝大部分广播包都成功到达邻居，网络可靠性高；时延数值反映了广播从发送到到达的时间开销，可用于推断MAC层时隙或处理延迟。

关键代码内容解释

在阅读上述程序和脚本实现时，有一些关键代码内容和机制值得用简明语言澄清：

- **定时器驱动发送机制：** `RadioCountToLedsC` 模块利用周期定时器实现持续发送。在 `MilliTimer.startPeriodic(250)` 设定后，每250ms触发一次 `fired` 事件。代码中直接在 `fired` 事件里执行发送逻辑，而非派发一个独立任务，这是因为发送操作足够快且TinyOS允许在事件中调用非阻塞发送命令。为了防止前一次发送未完成就重入下一次发送，代码使用了 `locked` 标志作为简单互斥锁：在发送开始时设锁，在 `sendDone` 时解锁。如果锁未解，新的 `fired` 事件只会立即返回不发送。这一机制保证了**不会有两个消息同时争用发送资源**，符合TinyOS单线程架构下对并发的控制要求。
- **消息结构与地址：** 本实验的数据包很简单，仅包含一个16位计数。在NesC中用 `nx_struct radio_count_msg` 定义，并通过 `AMSenderC(AM_RADIO_COUNT_MSG)` 和 `AMReceiverC(AM_RADIO_COUNT_MSG)` 将发送、接收接口绑定到该消息类型ID。发送时使用 `AM_BROADCAST_ADDR` 作为目的地址，实现广播。TinyOS主动消息层会自动将目的地址填为0xFFFF（广播）并在空中发送，邻居节点的接收组件在收到后都会触发相应类型的 `Receive.receive`。因此，无需在应用层逐个指定接收节点，广播由底层负责泛发。这种方式简化了多节点通信的代码实现。
- **调试日志时间戳：** 通过 `sim_time()` 获取的tick可以精确表示事件时间。`test.py` 定义 `TICKS_PER_SEC = 4000000.0` 用于换算tick到秒（TinyOS仿真默认4MHz时钟）。日志中记录的时间值很大，比如 `2441506351`，但这些只是相对于模拟时钟起点的计数。计算时延时，脚本用接收tick减发送tick，然后除以 `TICKS_PER_SEC` 得到秒，再乘1000得到毫秒。整个计算在代码中是：`delay_ms = (recv_tick - send_tick) / TICKS_PER_SEC * 1e3`。这样即可将两个事件的时间差转换为实际的毫秒级延迟。
- **正则表达式匹配与数据配对：** 仿真脚本使用了正则表达式从日志文本中提取所需信息。例如 `SEND_RE = re.compile(r"packet sent\\.s+counter=(\\d+)\\.s+time=(\\d+)")` 用于匹配“packet sent. counter=X time=Y”格式，并抓取X和Y两个数字。`RECV_RE` 类似匹配“received packet...”行。通过在循环读取每行日志时应用这些正则，脚本分别识别出发送事件和接收事件，并获取其中的counter值和时间戳。然后脚本将相同counter的发送/接收组合起来计算延迟。这种基于序号的配对假设每个数据包都有一个唯一的计数标识。在本实验中，由于每个节点有独立计数且不同节点可能产生重复的计数值（例如每个节点都发过“counter=1”），单靠counter可能会误配。因此脚本采取先进先出的队列策略：按照日志时间顺序，将发送事件依次入队，当出现接收事件且对应counter队列不空时，就出队一个发送用于匹配。这样确保每条接收最多对应之前最近的一次发送。而对于同一发送被多个节点收到的情况，只有第一次接收配对会消耗发送记录，其余接收将因队列已空而无法配对计算延迟（但仍会记入

总接收数以影响PDR计算)。这个实现权衡了复杂度和准确性,在节点数量不多且流量不大的情况下能够可靠反映总体趋势。

- **收包率计算细节:** 根据实验要求,收包率=PDR定义关注“请求-回应”关系。然而在广播情形下,一个请求会有多个回应,因此直接用接收数/发送数会超过100%。脚本通过计算理论**期望接收数**来解决这个问题:对于N个节点网络,每次发送有N-1个可能回应方,因而将发送总数乘以(N-1)得到如果毫无丢包理论上应收到的总包数。然后用实际收到的总数除以期望值,得到0~1的比率,再换算成百分比。这实际上计算的是**平均链路交付率**:即所有节点对之间成功传递的比例。例如上例中 $180/200=0.9$ 即90%。这种算法假设所有节点之间拓扑上互为邻居。如果某些节点之间本就无直接链路(例如拓扑矩阵中增益为 $-\infty$),期望值计算可能需要调整。不过本实验拓扑是完全连通的(三节点都有互相的链路配置),因此上述公式适用。输出的PDR百分比越接近100%,表示网络包的丢失率越低,可靠性越好。

通过上述分析,我们梳理了实验的完整流程和核心程序实现。从准备环境、编写仿真脚本、运行仿真获取日志,到最终根据日志提取结果,整个过程验证了TinyOS无线节点间**单跳广播通信**的功能,并通过统计**收包率**和**时延**量化了网络性能表现。本实验的代码相对简洁,但涵盖了TinyOS应用开发中的关键要点,包括**定时器驱动**、**无线收发**、**调试日志**、**仿真配置和结果分析**等,为理解传感器网络协议的实现与评价打下基础。 2

1 2 3 4 实验要求.doc
file:///file-Sx6jNDzZxMPcWoV2UhsCsr